

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería



Test Automatizados

Genser Catalán 23401, Fernando Ruíz 23065, Hugo Barillas 23306, Jose López 23773

Link al documento: [Ir a documento](#)

Link repositorio GITHUB: [Ir a github](#)

Erick Marroquín

Ingeniería en Software 1 – CC3090

Guatemala, 2025

Investigación de tecnologías

Frontend:

Para las pruebas del frontend, evaluamos varias tecnologías, principalmente Cypress y Vitest. Cypress destaca por permitir probar cómo los usuarios interactúan con la aplicación en un navegador, mostrando paso a paso la ejecución de las pruebas y ayudando a identificar errores de manera visual. Además, permite probar flujos completos de la aplicación y gestionar datos de prueba de forma sencilla.

Vitest, por su parte, es más reciente pero muy rápido y fácil de integrar, especialmente en proyectos contruidos con Vite. También permite probar componentes y la interacción con el DOM, lo que lo hace interesante para analizar la calidad del frontend. Comparando estas herramientas, queríamos entender cuál nos ofrecería una mejor combinación de velocidad, facilidad de uso y cobertura de pruebas para nuestro proyecto.

Backend:

En el backend, consideramos herramientas como Jest y Vitest para evaluar su capacidad de probar la lógica del backend en las APIs. Jest es una opción madura y ampliamente usada, con soporte para simulaciones, pruebas paralelas y documentación amplia, lo que facilita revisar funciones críticas en las varias funcionalidades del proyecto.

Vitest, aunque más reciente, ofrece ejecución rápida y flexibilidad, y también permite probar funciones de backend y APIs con ESM y fixtures. Analizamos estas tecnologías para ver cuál nos permitiría detectar errores de manera eficiente y mantener el backend estable mientras desarrollamos nuevas funcionalidades.

Tecnologías elegidas

Después de analizar varias opciones para las pruebas automatizadas, decidimos usar Vitest para el frontend y SuperJest para el backend. Elegimos Vitest porque se integra muy bien con Vue, que es la tecnología principal de nuestro proyecto, y permite realizar pruebas rápidas de componentes e integración de manera sencilla.

Para el backend optamos por SuperJest, ya que se adapta perfectamente a Express, nuestro framework de servidor, y ofrece funciones confiables para probar APIs y la lógica del servidor. En ambos casos, la compatibilidad con las tecnologías que estamos usando fue el factor principal para tomar la decisión, asegurando que las pruebas automatizadas se puedan implementar de forma eficiente y sin problemas de integración.

Videos de pruebas

Pruebas Frontend

1. HeaderComponent - Prueba funcional

Este test monta el header con un router en memoria y verifica el estado visual del enlace activo. Primero comprueba que al iniciar en /dashboard el link correspondiente se marca

como activo; luego navega a /productos y valida que el activo cambie a ese enlace y que /dashboard deje de estar activo.

Video: <https://youtu.be/xK2fEHuZCHU>

2. TallasModal – Prueba funcional

Este test valida el comportamiento del modal de tallas: cuando show=true renderiza la lista con los datos correctos; al hacer clic en los controles de cierre emite el evento close; y cuando show=false no muestra contenido.

Video: <https://youtu.be/VYG4fe41lgE>

3. Tabla Productos – Prueba de integración

Esta prueba se encarga de comprobar que la tabla de productos funcione bien, mostrando los datos como debe, aplicando los estilos según el stock y comunicándose correctamente con su componente principal. También revisa que las acciones del usuario, como seleccionar productos o activar el modo de eliminación, se realicen sin problemas

Video: <https://youtu.be/RDiZWzypUKc>

Pruebas Backend

1. Cliente test – Smoke test

Este smoke test examina que la ruta /clientes de tu aplicación Express esté accesible y devuelva una respuesta en formato JSON, con un código de estado válido (ya sea 200 OK si todo funciona, o 500 en caso de error interno). Para lograrlo, se simula el middleware de autenticación y la conexión a la base de datos con mocks, de modo que la prueba no depende de servicios externos

Video: [Smoke test cliente / inventario](#)

2. Inventario test – Smoke test

verifica que la ruta /inventory del backend funcione en condiciones mínimas sin depender de la base de datos real, del middleware de roles ni de los servicios de sockets (todos se reemplazan por mocks). En concreto, examina que al hacer un GET /inventory la aplicación responda con un código 200, un objeto JSON que contenga la propiedad success: true y un campo data que sea un arreglo. Con esto se asegura que la ruta existe, responde correctamente y mantiene el formato esperado, dejando para pruebas más profundas la lógica de negocio o conexión real a la base de datos

Video: [Smoke test cliente / inventario](#)

3. Auth test – smoke test

Envía peticiones a los endpoints de autenticación (/login, /logout, /forgot-password) y confirmar que devuelven algún código de estado válido. No se valida la lógica

completa ni los datos de la base, solo que el servidor arranque correctamente y los endpoints contesten

Video: [Auth smoke test](#)

Conclusiones

La implementación de pruebas automatizadas en el proyecto permitió garantizar la calidad tanto en el frontend como en el backend, asegurando que las funcionalidades críticas respondan de forma correcta y consistente. Tras la investigación y comparación de distintas tecnologías, se determinó que Vitest era la mejor opción para el frontend por su integración con Vue y su rapidez en pruebas de componentes, mientras que SuperJest resultó ideal para el backend gracias a su compatibilidad con Express y su capacidad para validar rutas y lógica de negocio mediante mocks.

Los casos de prueba realizados demostraron que el sistema puede ser evaluado sin depender de servicios externos, lo que facilita la detección temprana de errores y la validación de los flujos principales de la aplicación. Con esto, se sientan bases sólidas para continuar el desarrollo de nuevas funcionalidades con menor riesgo y mayor confianza en la estabilidad del sistema.

Referencias

- *Cypress.io vs Vitest comparison of testing frameworks*. (n.d.).
https://knapsackpro.com/testing_frameworks/difference_between/cypress-io/vs/vitest
- Tozzi, C. (2023, 20 septiembre). Vitest vs. Jest: Differences, Benefits, and Challenges. *SauceLabs*. <https://saucelabs.com/resources/blog/vitest-vs-jest-comparison>
- Souza, J. (2024, April 26). *Cypress: la herramienta que automatiza tus pruebas y garantiza la calidad de tus proyectos*. Itequia. <https://itequia.com/es/cypress-la-herramienta-que-automatiza-tus-pruebas-y-garantiza-la-calidad-de-tus-proyectos/>